DISEÑO DE UN SISTEMA RAG

POR: ELISA RINA CONDORCALLO QUEQUEJANA

CASO: CNN NOTICIAS

PROBLEMA

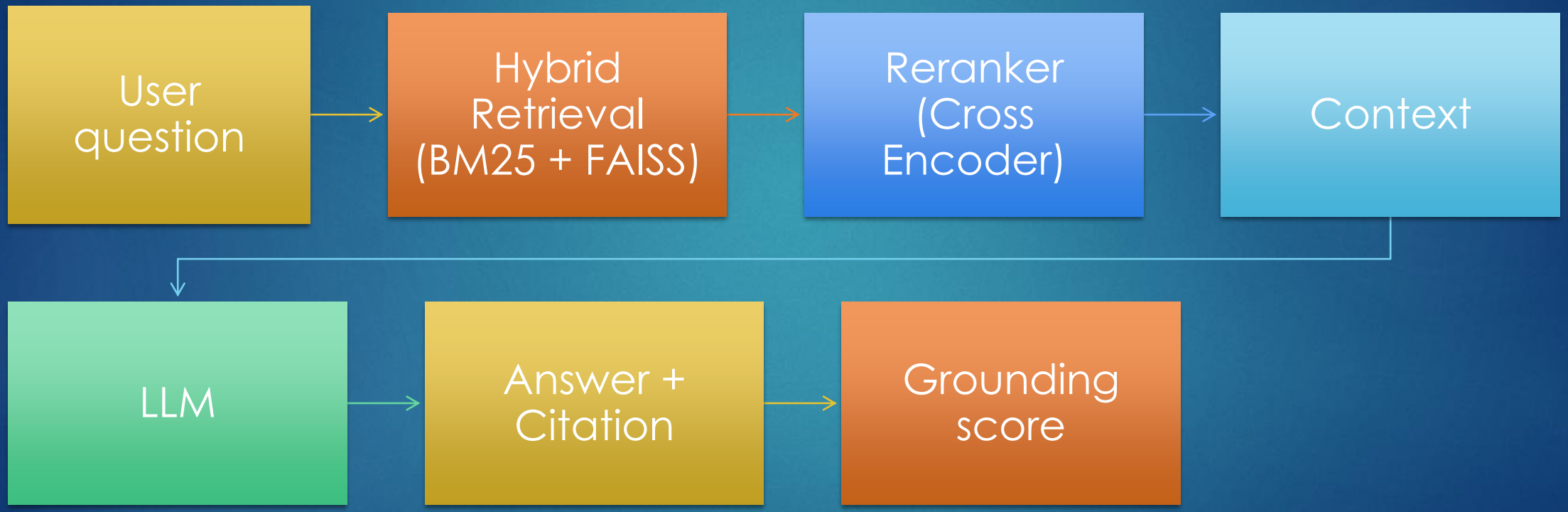
- Los modelos de lenguaje presentan un problema importante **Halluciations**

Inventar información

Responder sin evidencia

Generar respuestas incorrectas

ARQUITECTURA DEL SISTEMA RAG



Dataset y preparación de datos

- Se descarga el dataset **CNN/DailyMail**, una colección extensa de artículos de noticias que servirá como base de conocimiento del sistema.
- Características:
  - Artículos reales de noticias
  - Se utilizaron **200 documentos**
- Proceso:

```python
def chunk_text(text, size=400, overlap=100):
    chunks = []
    start = 0

    while start < len(text):
        chunk = text[start:start+size]
        chunks.append(chunk)
        start += size - overlap

    return chunks
```

El **overlap evita perder información** entre cortes y mantiene el contexto local en cada fragmento.

FAISS + Reranker

- Se implementó **Hybrid Retrieval**:
- **FAISS**
  - Búsqueda semántica con embeddings.
  - Modelo: **all-MiniLM-L6-v2**
  - Indexación: FAISS HNSW (32 conexiones)
- **BM25**
  - Búsqueda por palabras clave. Que mejora recall.
  - Reranker: **Modelo cross-encoder/ms-marco-MiniLM-L-6-v2**
- El **Reranker** (Modelo **cross-encoder/ms-marco-MiniLM-L-6-v2**) ordena los documentos por relevancia y se quedan con los **top 5**, mejorando significativamente la calidad del contexto.

Significado

```python
#Funcion Retrieve BM25
def retrieve_bm25(query, k=5):

    tokenized_query = query.split()

    scores = bm25.get_scores(tokenized_query)

    top_idx = np.argsort(scores)[::-1][:k]

    return [chunks[i] for i in top_idx]
```

Exactitud

```python
#Funcion hybrid retrieve:
def hybrid_retrieve(query, k=10):

    # Retrieval semántico (FAISS)
    docs_faiss = retrieve(query, k)

    # Retrieval léxico (BM25)
    docs_bm25 = retrieve_bm25(query, k)

    # Combinamos resultados
    combined = list(set(docs_faiss + docs_bm25))

    return combined
```

Precisión final

```python
#Cargar reranker CROSS ENCODER
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
```

Mostrar salida oculta

```python
#Funcion Reranking:
def rerank(query, docs, top_k=3):
    pairs = [[query, doc] for doc in docs]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
    return [doc for doc, score in ranked[:top_k]]
```

```python
#docs = retrieve(query, k=10)
docs = hybrid_retrieve(query, k=10)
reranked_docs = rerank(query, docs, top_k=5)
context = "\n".join(reranked_docs)
```

Hallucination + Grounding

- **1.Citation per sentence**
  - Cada oración de la respuesta tiene referencia:

Trazabilidad completa: mostrar de qué documento salió cada frase.

1. **Dividir respuesta**
   Separar en oraciones individuales
2. **Comparar embeddings**
   Calcular similitud con documentos
3. **Asignar fuente**
   Etiquetar documento más parecido

```
Climate change is caused by greenhouse gases [Doc2]
These gases trap heat in the atmosphere [Doc1]
```

Cada oración se etiqueta con su **documento fuente**, mejorando la transparencia y verificabilidad de la información.

```python
#Busqueda de chunk parecido
def cite_sentences(answer, docs):

    sentences = split_sentences(answer)
    sent_emb = embedder.encode(sentences)
    doc_emb = embedder.encode(docs)

    citations = []

    for i, s_emb in enumerate(sent_emb):
        scores = np.dot(doc_emb, s_emb)
        best_doc = np.argmax(scores)
        citations.append(f"{sentences[i]} [Doc{best_doc+1}]")

    return "\n".join(citations)
```

```python
def hallucination_guard(answer, context):
    hits = sum(1 for s in answer.split() if s.lower() in context.lower())
    return hits / max(len(answer.split()),1)
```

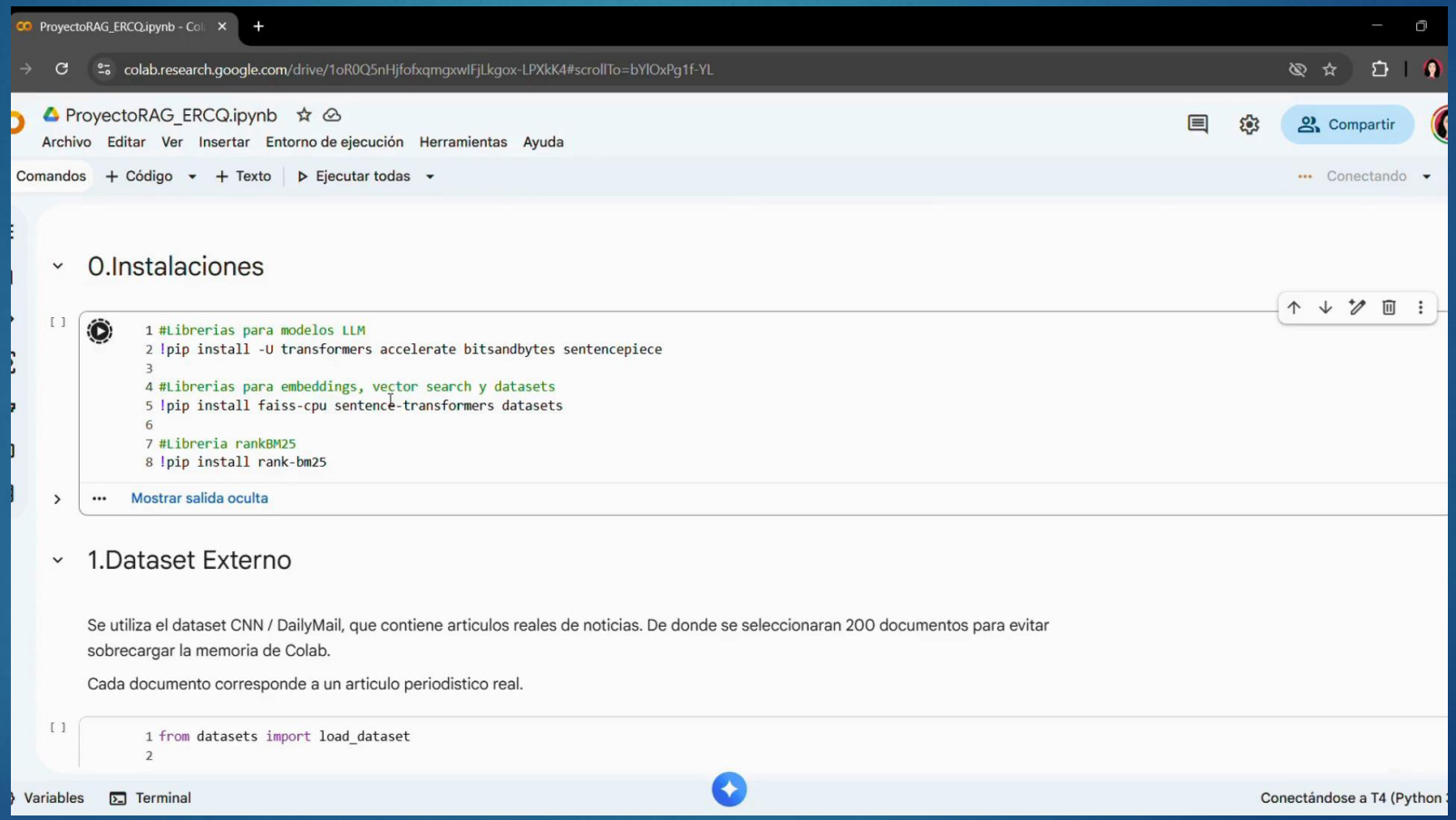
```python
def grounding(answer, retrieved_chunks):
    ctx = " ".join(retrieved_chunks)
    return hallucination_guard(answer, ctx)

```

- 2.**Grounding Score**
  - Mide cuánto de la respuesta está basado en el contexto.
  - Rango (**0 : posible alucinación; 1 : completamente fundamentado)**

Demo

Demostración del sistema:



Resultados

- Se evalua el sistema RAG, definiéndose un evaluation_set con preguntas y respuestas de referencia.
- Resultados obtenidos:
  - **Grouding Score promedio** = 0.85

    Alta fundamentación en documentos.

    Baja probabilidad de alucinaciones
- ROUGE-1 -> 0.03
- ROUGE-L -> 0.03
  - Baja similitud textual con referencia por el parafraseo usado.

El modelo usa bien el contexto, reduce alucinaciones y parafrasea, logrando respuestas **correctas y fundamentadas**, aunque no idénticas.

```python
#Creamos algunas preguntas de evalución
evaluation_set = {
    "What causes climate change?":
        "Climate change is mainly caused by greenhouse gases in the atmosphere.",

    "What are greenhouse gases?":
        "Greenhouse gases trap heat in the Earth's atmosphere.",

    "Why is global warming dangerous?":
        "Global warming can increase sea levels and extreme weather."
}
```

```python
#Creamos el evaluador
scorer = rouge_scorer.RougeScorer(['rouge1','rougeL'], use_stemmer=True)
```

```python
query = "What causes climate change?"
```

```python
for query, reference_answer in evaluation_set.items():

    # RETRIEVE
    #docs = retrieve(query, k=10)
    docs = hybrid_retrieve(query, k=10)

    # RERANK
    reranked_docs = rerank(query, docs, top_k=5)

    # CONTEXT
    context = "\n".join(reranked_docs)

    # LLM
    answer = ask_llm(context, query)

    # CITATION PER SENTENCE
    cited_answer = cite_sentences(answer, reranked_docs)

    # GROUNDING
    grounding_score = grounding(answer, reranked_docs)

    # ROUGE
    rouge_scores = scorer.score(reference_answer, answer)

    print("="*60)
    print("QUESTION:", query)

    print("\nANSWER:")
    print(cited_answer)

    print("\nGrounding score:", round(grounding_score,2))

    print("ROUGE-1:", round(rouge_scores['rouge1'].fmeasure,2))
    print("ROUGE-L:", round(rouge_scores['rougeL'].fmeasure,2))
```

```
Question:
What causes climate change [Doc5]
assistant
Global warming is caused by an increase in the concentrations and the intensity
This is primarily due to the accumulation of greenhouse gases (carbon dioxide) i
These gases trap heat from the sun's radiation and prevent it from escaping back
As a result, many global warming experts believe that climate change will lead t

Grounding score: 0.88
ROUGE-1: 0.04
ROUGE-L: 0.04
```

```
Question:
What are greenhouse gases [Doc2]
assistant
Greenhouse gases are natural compounds that absorb and retain heat in Earth's at
They include:

  * Methane: A greenhouse gas produced by the breakdown of organic matter in liv
Its concentration has risen by 151% since pre-industrial times [Doc3]
* Nitrous oxide: A greenhouse gas emitted by agricultural practices, such as the
It accounts for around 18% of the total global atmospheric nitrous oxide concent
* Carbon dioxide: A greenhouse gas produced by the combustion of fossil fuels (c
Its concentration has increased by 35% since the dawn of the industrial age [Doc
* Water vapor: A greenhouse gas that occurs naturally due to the cooling of wate
It contributes about 7% to the Earth's total greenhouse gas content [Doc1]
These gases play a crucial role in regulating the Earth [Doc1]

Grounding score: 0.83
ROUGE-1: 0.03
ROUGE-L: 0.03
```

```
Question:
Why is global warming dangerous [Doc5]
assistant
Global warming is dangerous because it has the potential to alter the world's cl
According to many global warming experts, including Stephen Scince, some warming
These greenhouse gases, such as carbon dioxide (CO2), contribute to global warmi
As these gases remain in the atmosphere, they lead to more frequent and intense
Moreover, the increase in atmospheric CO2 [Doc5]

Grounding score: 0.85
ROUGE-1: 0.03
ROUGE-L: 0.03
```

Conclusiones

- En conclusión, el **sistema RAG** permite mejorar la **confiabilidad** de los **modelos de lenguaje** al obligarlos a responder usando **documentos recuperados.**
- Además, los mecanismos de **citación** y **grounding** permiten **verificar la fuente** de la información.